DevOps Task Manager Application using React and Node.js

A course project report submitted in partial fulfilment of the requirement

of

DEVOPS AND FULLSTACK

by

T. Goutham Reddy (2303A51345)

K. Charan Yadav (2303A52369)

Under the guidance of

Dr. N.VENKATESH

Asst. Professor, Department of CS & AI

SR University, Warangal

APRIL-2026

ABSTRACT

The main aim of this project is to develop a DevOps-based full-stack Task Manager application that helps users manage their daily tasks efficiently. This application allows users to add, view, and delete tasks through a simple and user-friendly interface.

The frontend is developed using React.js, and the backend is implemented using Node.js and Express.js. The communication between frontend and backend is achieved through RESTful APIs.

This project demonstrates basic DevOps practices such as modular architecture, separation of frontend and backend, and readiness for continuous integration and deployment. The system is simple, efficient, and scalable for future enhancements.

CONTENTS

Chapter No.	Title	Page No.
ABSTRACT		*ii*
1	**INTRODUCTION**	01
	1.1 About the project	01
	1.2 Objectives	02
2	**LITERATURE REVIEW**	
	2.1 Overview	03
	2.2 Related Studies and Existing Solutions	05
	2.3 Tools and Technologies Used	07
3.	**METHODOLOGY**	
	3.1 Data Collection and Description	08
	3.2 Data Preprocessing and Cleaning	10
	3.3 Analytical and Visualization Techniques	12
4	**RESULTS AND DISCUSSION**	
	4.1 Data Analysis and Key Findings	14
	4.2 Visualization and Interpretation of Results	16
	4.3 Insights and Discussion	18
5	**CONCLUSION AND FUTURE WORK**	
	5.1 Summary of Findings	19
	5.2 Conclusions Drawn	20
	5.3 Recommendations and Future Scope	20

INTRODUCTION

1.1 About the Project

The DevOps Task Manager Application is a full-stack web application designed to manage daily tasks efficiently. It allows users to add, view, and delete tasks using an interactive user interface.

The application is built using React.js for the frontend and Node.js with Express.js for the backend. The frontend communicates with the backend using APIs to perform operations such as adding and deleting tasks.

This project helps in understanding full-stack development and the role of DevOps in modern software systems.

1.2 Objectives

- To develop a full-stack web application
- To implement communication between frontend and backend
- To understand DevOps concepts
- To create a simple task management system
- To gain practical knowledge of modern technologies

CHAPTER 2: LITERATURE REVIEW

DevOps and Full-Stack Application Development

DevOps has emerged as a transformative approach in modern software engineering by bridging the traditional gap between development and operations teams. The concept gained prominence around 2009 and emphasizes collaboration, automation, continuous integration, and continuous deployment throughout the software development lifecycle. Researchers such as Humble and Farley (2010) highlighted that continuous delivery practices significantly reduce deployment risks while improving software reliability and release frequency. DevOps integrates development, testing, deployment, monitoring, and feedback mechanisms into a continuous and automated pipeline, thereby enhancing efficiency and reducing time-to-market.

Continuous Integration and Continuous Deployment (CI/CD) are core components of DevOps practices. CI/CD pipelines automate code building, testing, and deployment processes, enabling rapid and reliable software releases. Tools such as Jenkins, GitHub Actions, and GitLab CI/CD are widely adopted for implementing automated workflows. Literature indicates that CI practices help detect defects early, minimize integration conflicts, and enhance collaborative development. Automated deployment further ensures consistency across staging and production environments.

Containerization has significantly influenced DevOps implementation by enabling consistent application environments. Docker introduced lightweight containers that package applications along with their dependencies, ensuring portability across different systems. This approach reduces configuration mismatches and improves scalability. For orchestration and large-scale container management, Kubernetes provides automated scaling, load balancing, and self-healing capabilities. Studies suggest that container-based deployment improves resource utilization and simplifies microservices architecture implementation.

Version control systems form the backbone of collaborative software development in both DevOps and full-stack environments. Git enables distributed source code management and supports branching, merging, and pull request workflows. Research demonstrates that structured version control strategies

enhance maintainability, reduce development conflicts, and improve overall software quality.

Full-stack development integrates both frontend and backend technologies to build complete web applications. On the frontend, frameworks such as React and Angular facilitate dynamic user interfaces and component-based architectures. On the backend, technologies like Node.js and Express.js enable scalable server-side development and RESTful API integration. Literature suggests that full-stack frameworks enhance development productivity, modular design, and seamless client–server communication. The integration of frontend and backend technologies within a unified ecosystem accelerates application delivery and supports agile methodologies.

Database integration is another critical aspect of full-stack systems. Relational databases such as MySQL provide structured data management and strong consistency, while NoSQL databases such as MongoDB offer flexible schemas and horizontal scalability. Comparative studies highlight that NoSQL databases are often preferred in JavaScript-based full-stack ecosystems due to their schema flexibility and JSON-like document storage format.

Infrastructure as Code (IaC) further strengthens DevOps maturity by automating infrastructure provisioning and configuration management. Tools like Terraform and Ansible allow developers to define infrastructure through code, improving reproducibility and reducing manual errors. Research indicates that IaC enhances deployment consistency and supports cloud-native application development.

Monitoring and observability are essential for maintaining reliability in production systems. Tools such as Prometheus and Grafana enable real-time performance tracking, log aggregation, and alert-based incident management. Literature emphasizes that proactive monitoring reduces downtime and improves system resilience.

Overall, existing studies demonstrate that the integration of DevOps practices with full-stack development significantly enhances software quality, deployment speed, scalability, and collaboration efficiency. The convergence of automation, containerization, CI/CD pipelines, infrastructure management, and monitoring represents a paradigm shift from traditional development models toward agile, cloud-native, and continuously evolving software systems.

CHAPTER 3: METHODOLOGY

3.1 System Design

The system is divided into frontend and backend. The frontend is responsible for user interaction, while the backend handles data processing and API requests.

3.2 Working Process

- User enters a task
- React sends request to backend
- Backend processes the request
- Task is stored and returned
- UI updates automatically

3.3 Technologies Used

The project uses React.js for frontend and Node.js with Express.js for backend. Axios is used to connect both parts of the application.

3.4 Tools and Technologies Used

- React.js for frontend development
- Node.js for backend development
- Express.js for API creation
- Axios for communication
- Git and GitHub for version control

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Results

The application was successfully developed and tested. Users can add, view, and delete tasks easily.

4.2 Analysis

The system performs efficiently and provides a smooth user experience. The communication between frontend and backend is working correctly.

4.3 Discussion

The project shows how full-stack development and DevOps practices can be used to build simple and scalable applications.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Summary

The project successfully demonstrates a full-stack Task Manager application using modern technologies.

5.2 Conclusion

The system is simple, user-friendly, and efficient. It provides a basic understanding of DevOps and full-stack development.

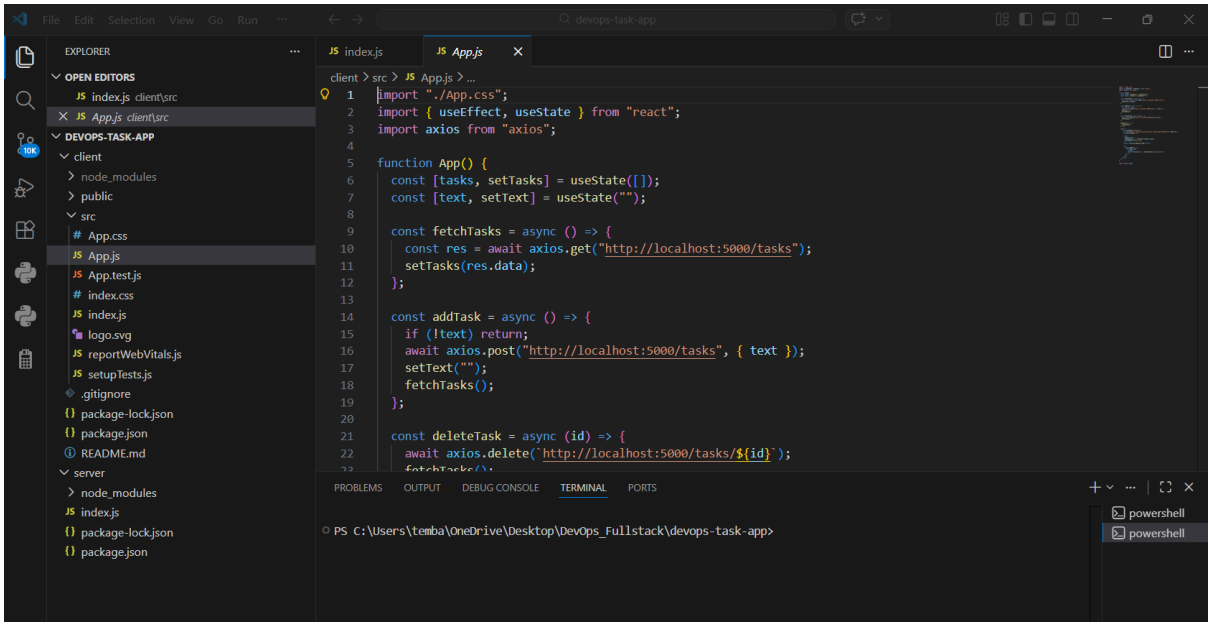
5.3 Future Scope

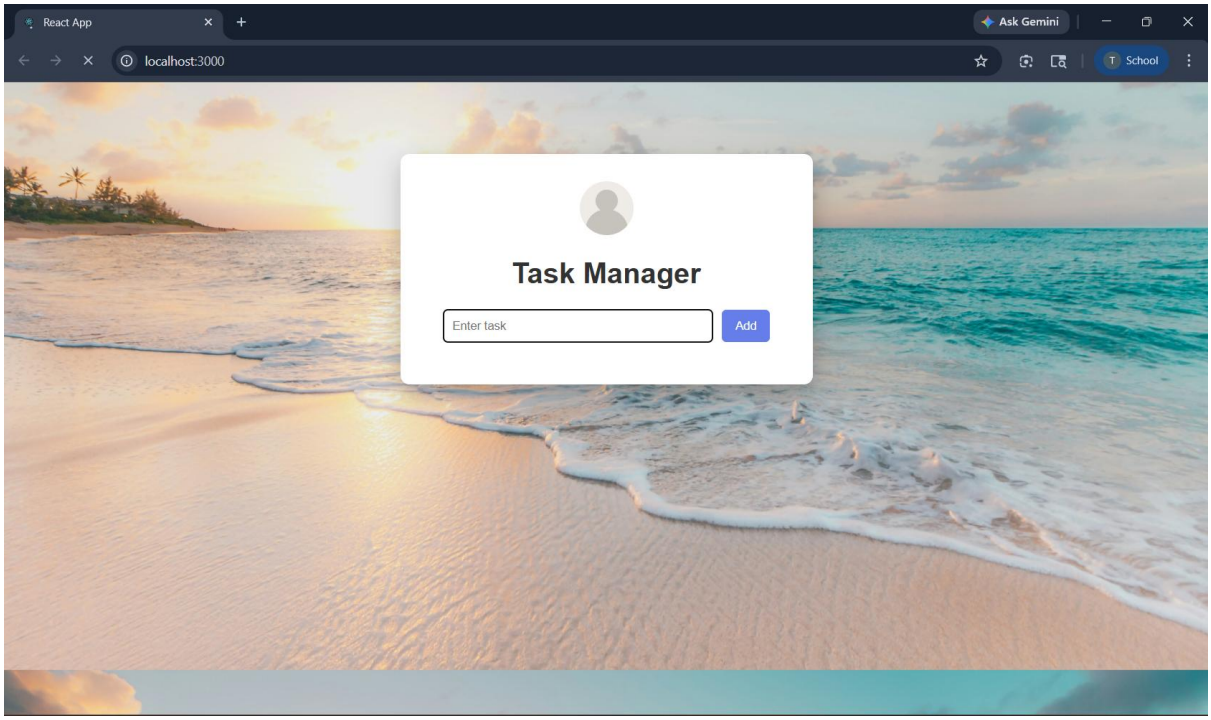
Add database (MongoDB)

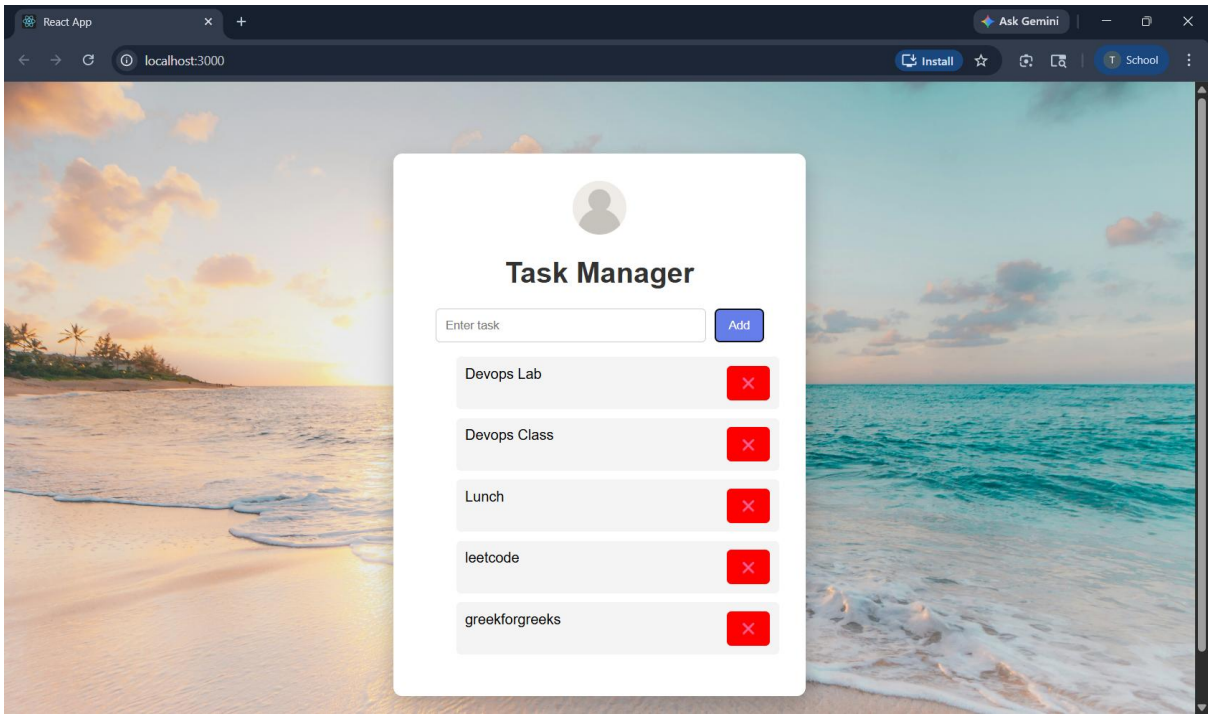
Add user login system

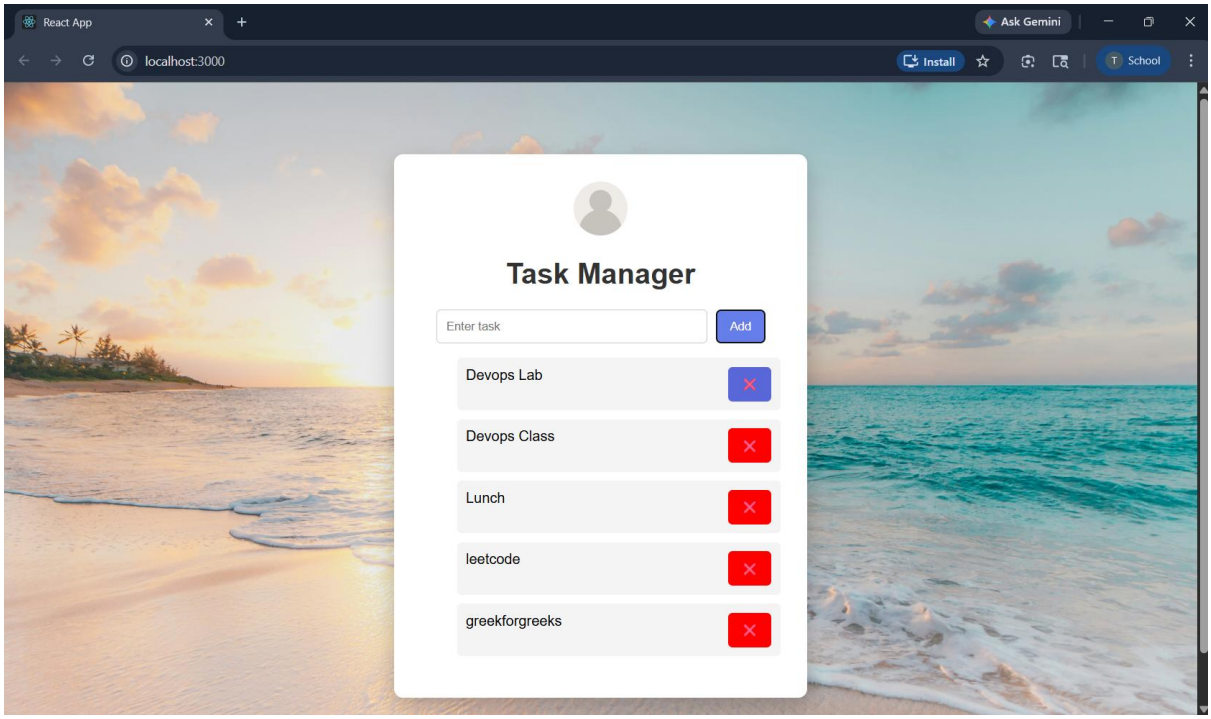
Deploy using Docker

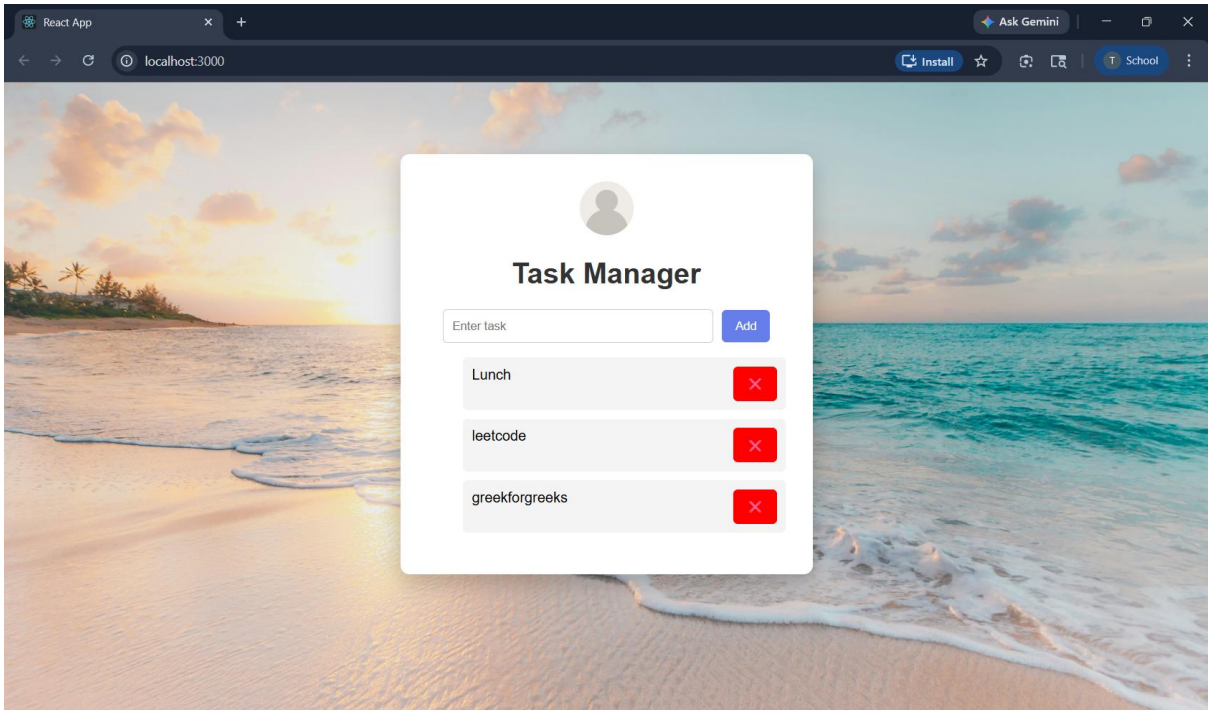
Improve UI design




React App
Ask Gemini
localhost:3000
School
Task Manager
Enter task
Add


React App
Ask Gemini
localhost:3000
Install
School
Task Manager
Enter task
Add
Devops Lab
Devops Class
Lunch
leetcode
greekforgreeks


React App
Ask Gemini
localhost:3000
Install
School
Task Manager
Enter task
Add
Devops Lab
Devops Class
Lunch
leetcode
greekforgreeks


React App
Ask Gemini
localhost:3000
Install
School
Task Manager
Enter task
Add
Lunch
leetcode
greekforgreeks